

2. Последовательный поиск

Цель комплекса лабораторных работ – сравнить и проанализировать алгоритмы последовательного поиска в неупорядоченной и упорядоченной последовательности данных.

2.1. Теоретическая часть

Поиском называется процедура выделения из множества записей его подмножества, записи которого удовлетворяют заданному условию [3]. Множество записей может храниться в массиве, связанном списке, дереве или графе.

Здесь и далее под записью понимается как отдельное поле, содержащее одно значение, так и структура любой допустимой степени сложности.

В дальнейшем будем считать, что имеется множество из N записей и необходимо определить положение записи, удовлетворяющей некоторому заданному условию. Предположим, что каждая запись содержит специальное поле, которое называется **ключом**.

В алгоритмах поиска обычно задан **аргумент** поиска K , и задача состоит в нахождении записи, имеющей K своим ключом. Существуют две возможности окончания поиска: либо поиск оказался **удачным**, т.е. была найдена запись с ключом K , либо он оказался **неудачным**, т.е. ключ K не был найден ни в одной записи.

Хотя целью поиска является информация, содержащаяся в записи, в дальнейшем будем рассматривать алгоритмы, в которых обрабатываются только ключи.

Во многих задачах поиск требует наибольших временных затрат, поэтому замена «плохого» метода поиска на «хороший» часто ведет к существенному увеличению скорости работы.

Нередко удается так *организовать* данные, что поиск значительно убыстряется или почти полностью исключается.

Эффективность различных алгоритмов поиска оценивается количеством сравниваемых пар ключей, необходимым для выполнения условия поиска. Обычно оценивается **среднее** и **максимальное** количество сравнений.

Рассмотрим несколько способов *организации и поиска* данных. При этом основное внимание будет уделено *сравнительному* анализу изучаемых алгоритмов, их сложностным и временным характеристикам.

2.1.1. Неоптимальный последовательный поиск

Если нет никакой дополнительной информации о способе хранения данных, то вероятнее всего данные расположены в произвольном порядке. В этом случае очевидный подход – простой последовательный просмотр массива. Сформулируем алгоритм более точно [4].

Пусть имеется массив записей с ключами $A_1, A_2, \dots, A_N$. Необходимо найти запись с заданным ключом K .

Алгоритм А.

1. $i = 1$.
2. Если $K = A_i$, алгоритм заканчивается успешно.
3. Иначе $i = i + 1$.
4. Если $i \leq N$, то вернуться к шагу 2.
5. Иначе алгоритм заканчивается неудачно.

Условия окончания поиска следующие:

1. Элемент найден, т.е. обнаружен ключ $A_i = K$.
2. Все записи просмотрены, и совпадение не обнаружено.

Ясно, что **максимальное число сравнений** при удачном и неудачном поиске равно N . **Среднее число сравнений** при удачном поиске равно $(N + 1) / 2$. Для этого метода поиска, как среднее, так и максимальное время поиска пропорционально N , т.е. **сложность рассмотренного алгоритма** равна $O(N)$.

2.1.2. Оптимальный последовательный поиск

Приведенный выше алгоритм при всей своей очевидности не является, однако, оптимальным. В алгоритме неоптимального последовательного поиска имеются две проверки: **на совпадение ключей** ($K = A_i$) и **на выход значения индекса за пределы массива** ($i \leq N$).

Если гарантировать, что ключ всегда будет найден, то без второго условия (выход индекса за границу массива) можно обойтись. Для этого достаточно в конце ($N + 1$ -й элемент) массива поместить дополнительный элемент, присвоив ему значение искомого ключа K . Назовем такой вспомогательный элемент **барьером**, так как он предотвращает выход за пределы массива [3].

Таким образом, ключ всегда будет найден в записи с индексом i . При $i = N + 1$ (индекс достиг конца массива) ключ не найден (т.е. найден только в барьере). Поиск неудачен. В противном случае – поиск удачен.

Такой подход позволяет исключить в среднем $(N + 1) / 2$ сравнений.

Алгоритм, реализующий описанный выше метод поиска, называется **алгоритмом оптимального последовательного поиска**.

Алгоритм В.

1. $A_{N+1} = K$.
2. $i = 1$.
3. Если $K = A_i$, на шаг 5.
4. Иначе $i = i + 1$, на шаг 3.
5. Если $i = N + 1$, то поиск неудачен (ключ найден в барьере), иначе – поиск удачен (ключ найден в i -ом элементе массива).

2.1.3. Последовательный поиск в упорядоченном массиве

Если даже массив упорядочен, то и в этом случае иногда целесообразно использовать последовательный поиск. Нет смысла программировать более сложные алгоритмы поиска или использовать для хранения данных более сложную структуру, если количество записей невелико либо поиск должен быть выполнен однократно.

Если известно, что ключи упорядочены (например, расположены по степени возрастания), то алгоритм последовательного поиска целесообразно несколько изменить.

Пусть имеется множество записей с ключами $A_1, A_2, \dots, A_N$, причем $A_1 \leq A_2 \leq \dots \leq A_N$. Необходимо найти запись с заданным ключом K . В целях увеличения скорости работы в конец массива помещается дополнительный элемент – **барьер**, значение которого должно быть *больше* значения искомого ключа K (например, $K + 1$).

В отличие от поиска в неупорядоченном массиве, просмотр элементов упорядоченного массива заканчивается в тот момент, когда первый раз выполнится условие $K \leq A_i$. Очевидно, что все элементы $A_{i+1}, \dots, A_n$ будут больше.

Например, если имеется упорядоченная последовательность 3, 5, 8, 10, 11, 15, 23, 30, 31, 36, 40, 44, ... и ключ поиска $K = 14$, то, как только A_i станет равно 15, очевидно, что дальнейший поиск в упорядоченном массиве не имеет смысла.

Если $K = A_i$ – поиск удачен. В противном случае искомого элемента нет в массиве.

Алгоритм С.

1. $A_{N+1} = K + 1$.
2. $i = 1$.
3. Если $K \leq A_i$, на шаг 5.

4. Иначе $i = i + 1$, на шаг 3.

5. Если $K = A_i$, то поиск удачен, иначе – поиск неудачен.

В случае удачного поиска алгоритм поиска в упорядоченном массиве **не лучше** алгоритма поиска в неупорядоченном массиве, однако **отсутствие нужного ключа** этот алгоритм позволяет обнаружить в среднем **в два раза быстрее**.

Таким образом, среднее число сравнений и при удачном, и при неудачном поиске в алгоритме последовательного поиска в упорядоченном массиве равно $(N + 1) / 2$.

2.2. Задания на лабораторные работы

Рассмотренный выше материал является основой для трех лабораторных работ, представляющих собой единый программный комплекс. Цель программного комплекса – сравнение двух пар алгоритмов: алгоритма *A* и алгоритма *B* для неупорядоченного массива, а также алгоритма *B* и алгоритма *C* для упорядоченного массива.

Каждая лабораторная работа рассчитана на одно занятие и выполняется от начала и до сдачи строго на практическом занятии в компьютерном классе каждым студентом индивидуально. Работа в бригадах запрещена. Единственный вариант успеть сделать задание за одну пару – готовиться к каждому занятию.

Для выполнения лабораторных работ по данной дисциплине можно использовать любое инструментальное средство, обладающее достаточно широкими возможностями разработки Windows-приложений, использующих элементы интерфейса Windows, включая формы, кнопки, поля для ввода числовых и текстовых значений и т.п.

В пособии в качестве примеров будут использоваться программы, выполненные в среде Visual Studio 2022 на языке программирования C#.

2.2.1. Задание на лабораторную работу № 2

Неоптимальный последовательный поиск

1. Разработать интерфейс программного комплекса лабораторных работ № 2–4. Пример формы приведен на рис. 6. Рисунок демонстрирует только функционал разрабатываемой программы. Ваш вариант должен предоставлять не меньшие возможности. Расположение компонентов, состав компонентов, оформление частей формы в вашем варианте может быть другим.

ЛР2-4

Лабораторная работа № 2

Неупорядоченный массив	Упорядоченный массив
Неоптимальный поиск Ключ 0 Время Индекс Оптимальный поиск Время Индекс Найти	Как в неупорядоченном Ключ 0 Время Индекс Как в упорядоченном Время Индекс Найти

Выход

Рис. 6. Форма «Последовательный поиск»

2. Запрограммировать алгоритм неоптимального последовательного поиска (алгоритм *A*) для неупорядоченного массива.

Массив из 1000000–10000000 элементов задавать с помощью датчика случайных чисел (в методе *Form1_Load*). В случае высокого быстродействия используемого для снятия временных характеристик компьютера поиск следует осуществлять многократно – 100–10000 раз – либо увеличить размерность исходного массива. Подобрать размер массива и, если требуется, количество зацикливаний таким образом, чтобы минимальное время поиска было не менее 100 тиков.

На форме предусмотреть поле для ввода искомого ключа. Выводить результат поиска (индекс найденного элемента/поиск неудачен), а также время поиска ключа.

Для задания неупорядоченного массива датчиком случайных чисел можно использовать следующий код:

```
Random rnd = new Random();
for (int i = 0; i < n; i++) mas[i] = rnd.Next(0, n).
```

Для вычисления времени выполнения блока кода можно использовать следующий подход:

```
int StartTime = Environment.TickCount;  
{  
Блок кода  
}  
int ResultTime = Environment.TickCount - StartTime;  
textBox1.Text = ResultTime.ToString();
```

Для получения максимального количества баллов лабораторная работа № 2 должна быть сделана и сдана на втором лабораторном занятии.

Контрольные вопросы

1. Дайте определение понятия «поиск».
2. Каково среднее количество сравнений при поиске в неупорядоченном массиве, если поиск неудачен?
3. Каково среднее количество сравнений при поиске в неупорядоченном массиве, если поиск удачен?

2.2.2. Задание на лабораторную работу № 3

Оптимальный последовательный поиск

1. Используя разработанную в лабораторной работе № 2 программу, запрограммировать алгоритм оптимального последовательного поиска (алгоритм **B**) для неупорядоченного массива.
2. Выполнить сравнение времени поиска алгоритмов неоптимального последовательного поиска (алгоритма **A**) и оптимального последовательного поиска (алгоритма **B**) для неупорядоченного массива.

Для получения максимального количества баллов лабораторная работа № 3 должна быть сделана и сдана на третьем лабораторном занятии.

Примерный результат работы программы приведен на рис. 7.

Лабораторная работа № 3	
Неупорядоченный массив	Упорядоченный массив
Неоптимальный поиск Ключ 30410729 Время 125 Индекс 65096822 Оптимальный поиск Время 109 Индекс 65096822 Найти	Как в неупорядоченном Ключ 0 Время Индекс Как в упорядоченном Время Индекс Найти
Выход	

Рис. 7. Результат лабораторной работы № 3

Контрольные вопросы

1. В чем преимущество оптимального последовательного поиска по сравнению с неоптимальным поиском?
2. Какое из сравнений в цикле неоптимального последовательного поиска отсутствует в цикле оптимального последовательного поиска и на каком основании его можно убрать из цикла?

2.2.3. Задание на лабораторную работу № 4

Последовательный поиск в упорядоченном массиве.

Сравнение алгоритмов последовательного поиска

1. Используя разработанную в лабораторных работах № 2, 3 программу, запрограммировать алгоритм оптимального последовательного поиска для упорядоченного массива (алгоритм **С**).
2. Сравнить работу алгоритмов оптимального поиска в неупорядоченном массиве (алгоритм **В**) и алгоритмов оптимального поиска в упорядоченном массиве (алгоритм **С**) для упорядоченного массива.

Упорядоченный массив из 1000000–10000000 элементов задавать с помощью датчика случайных чисел (в методе **Form1_Load**). В случае высокого быстродействия используемого для снятия временных характеристик компьютера поиск следует осуществлять многократно – 100–10000 раз – либо увеличить размерность исходного массива. Подобрать размер массива и, если требуется, количество зацикливаний таким образом, чтобы минимальное время поиска было не менее 100 тиков.

Внимание! Алгоритм **В** в программном комплексе используется дважды. В левой части – для сравнения с алгоритмом **А** при поиске в неупорядоченном массиве (лабораторная работа № 3), в правой – для сравнения с алгоритмом **С** при поиске в упорядоченном массиве (лабораторная работа № 4).

Примерный результат работы программы приведен на рис. 8.

Лабораторная работа № 4	
Неупорядоченный массив	Упорядоченный массив
Неоптимальный поиск Ключ 25801676 Время 203 Индекс Нет Оптимальный поиск Время 157 Индекс Нет Найти	Как в неупорядоченном Ключ 92065767 Время 78 Индекс 46035108 Как в упорядоченном Время 78 Индекс 46035108 Найти
Выход	

Рис. 8. Результат лабораторной работы № 4

Для получения максимального количества баллов лабораторная работа № 4 должна быть сделана и сдана на четвертом лабораторном занятии.

Пятое лабораторное занятие предназначено для сдачи долгов по предыдущим лабораторным работам.

Контрольные вопросы

1. В чем отличие алгоритма оптимального последовательного поиска в неупорядоченном массиве от алгоритма оптимального последовательного поиска в упорядоченном массиве?
2. Задан упорядоченный массив и ключ поиска ***K***, значение которого присутствует в массиве. Какой алгоритм (алгоритм ***B*** или алгоритм ***C***) быстрее найдет этот ключ?
3. Задан упорядоченный массив и ключ поиска ***K***, значение которого отсутствует в массиве. Какой алгоритм (алгоритм ***B*** или алгоритм ***C***) быстрее не найдет этот ключ?